

Federated Learning Algorithm on Image Data

Jannatul Feardous

*B.Sc., Electrical & Electronic Engineering ,
Rajshahi University of Engineering & Technology*

1. Introduction

The need for user data training is significant in technology development, necessitating the use of machine learning approaches. Conventional learning methods collect data from individual devices and send to a common server, which can harm data and slow. Federated Learning, a machine learning method, can help train data without sharing it with any common server, rather it uses local data to train a common model on their own platform or device. After training, corrected weights or model updates are sent to a central server for further development. This approach saves bandwidth and makes the process faster.

Before training globally in the central server, it assigns a model to locally to each device, which then trains the model using its own data with Stochastic Gradient Descent (SGD) algorithm. After this Each device sends back only the updates as weights, to the central server. The collected weights are averaged which helps to improve the model for training repeatedly until fully trained. In the paper datasets form MNIST, CIFER are used for learning even with non-independent and identically distributed data. It has been proved that, Federated Averaging works well even Non-IID and Unbalanced Data which are not evenly distributed or not from same probability distribution.

2. Code Breakdown

2.1. Initialization and On Local Device Training

```
import torch
from torch import nn, autograd
from torch.utils.data import DataLoader, Dataset
import numpy as np
import random
from sklearn import metrics
```

Here necessary utilities function named ***Dataloader*** and Datasets are loaded.

```
class DatasetSplit(Dataset):
    def __init__(self, dataset, idxs):
        self.dataset = dataset
        self.idxs = list(idxs)

    def __len__(self):
        return len(self.idxs)

    def __getitem__(self, item):
        image, label = self.dataset[self.idxs[item]]
        return image, label

class LocalUpdate(object):
    def __init__(self, args, dataset=None, idxs=None):
        self.args = args
        self.loss_func = nn.CrossEntropyLoss()
        self.selected_clients = []
        self.idr_train = DataLoader(DatasetSplit(dataset, idxs), batch_size=self.args['local_bs'], shuffle=True)

    def train(self, net):
        net.train()
        # train and update
        optimizer = torch.optim.SGD(net.parameters(), lr=self.args['lr'], momentum=self.args['momentum'])

        epoch_loss = []
        for iter in range(self.args['local_ep']):
            batch_loss = []
            for batch_idx, (images, labels) in enumerate(self.idr_train):
                images, labels = images.to(self.args['device']), labels.to(self.args['device'])
                net.zero_grad()
                log_probs = net(images)
                loss = self.loss_func(log_probs, labels)
                loss.backward()
                optimizer.step()
                if self.args['verbose'] and batch_idx % 10 == 0:
                    print('Update Epoch: {} [{} / {}] ({:.0f}%) \t loss: {:.6f}'.format(
                        iter, batch_idx + 1, len(images), len(self.idr_train.dataset),
                        100 * batch_idx / len(self.idr_train), loss.item()))
                batch_loss.append(loss.item())
            epoch_loss.append(sum(batch_loss) / len(batch_loss))
        return net.state_dict(), sum(epoch_loss) / len(epoch_loss)
```

Firstly, central server starts with an initial version of a machine learning model. So, Data from clients have to be get ready to participate in training without shared with other. **DatasetSplit** is set to the raw and initial datasets be prepared to be divided into batches with specified indices for each type data. Here the data types are images and they are labeled as per their indices. This is needed as before locally training; after that some portion of data are to be taken to make smaller batches for more efficient training in **dataloader**.

Next, **args** is a set of predefined values. Loss function is to determine the percentages of errors after training. The batch size to train the data is determined by '**local_bs**' value which is taken as 32, predefined as an element of **args**. Also, the data shuffling is set up so that all are more generalized and training is more accurate. The training method is set as SGD with learning rate (**lr**) and momentum given. In **self.ltr_train** the batches will be put and then all the batches of image data with their respective labels will be put in the device where the training is to be run. Then, iterations are to be run to find losses for each epoch of training and store them so that the losses can help improvement. Once a device finishes training locally with its data, it has to send over not the whole model, it sends only the updates (weights), the changes it made to the model. The server averages the updated models. The more data a device has, the more weight its updates are given in the averaging process. Here all these are defined under train function.

Here, the epochs, number of users, locally training epochs, locally training batch members, global training batch numbers, learning rate are set as 10, 100, 5, 32, 64 and 0.01 respectively. Also the image data are RGB as the number of channels are set to be 3. The other values also are set inside **args**.

2.2. Global Training on Server

```
# load dataset and split users
if args['dataset'] == 'mnist':
    trans_mnist = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])
    dataset_train = datasets.MNIST('./sample_data/', train=True, download=True, transform=trans_mnist)
    dataset_test = datasets.MNIST('./sample_data/', train=False, download=True, transform=trans_mnist)
    # sample users
    if args['iid']:
        dict_users = mnist_id(dataset_train, args['num_users'])
    else:
        dict_users = mnist_noniid(dataset_train, args['num_users'])
elif args['dataset'] == 'cifar':
    trans_cifar = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
    dataset_train = datasets.CIFAR10('./sample_data/', train=True, download=True, transform=trans_cifar)
    dataset_test = datasets.CIFAR10('./sample_data/', train=False, download=True, transform=trans_cifar)
    if args['iid']:
        dict_users = cifar_id(dataset_train, args['num_users'])
    else:
        exit('Error: only consider IID setting in CIFAR10')
else:
    exit('Error: unrecognized dataset')
img_size = dataset_train[0][0].shape

# build model
if args['model'] == 'cnn' and args['dataset'] == 'cifar':
    net_glob = CNNCIFAR(args=args).to(args['device'])
elif args['model'] == 'cnn' and args['dataset'] == 'mnist':
    net_glob = CMNMNIST(args=args).to(args['device'])
elif args['model'] == 'mlp':
    len_in = 1
    for x in img_size:
        len_in *= x
    len_in = len_in * 3
    net_glob = MLP(dim_in=len_in, dim_hidden=200, dim_out=args['num_classes']).to(args['device'])
else:
    exit('Error: unrecognized model')
print(net_glob)
net_glob.train()
```

```
# training
loss_train = []
cv_loss, cv_acc = [], []
val_loss_pre, counter = 0, 0
net_best = None
best_loss = None
val_acc_list, net_list = [], []

if args['all_clients']:
    print("Aggregation over all clients")
    w_locals = [w_glob for i in range(args['num_users'])]
for iter in range(args['epochs']):
    loss_locals = []
    if not args['all_clients']:
        w_locals = []
        m = max(int(args['frac'] * args['num_users']), 1)
        idxs_users = np.random.choice(range(args['num_users']), m, replace=False)
        for idx in idxs_users:
            local = LocalUpdate(args=args, dataset=dataset_train, idxs=dict_users[idx])
            w, loss = local.train(net=copied.deepcopy(net_glob).to(args['device']))
            if args['all_clients']:
                w_locals[idx] = copy.deepcopy(w)
            else:
                w_locals.append(copy.deepcopy(w))
            loss_locals.append(copy.deepcopy(loss))
```

From MNIST source, sample data, which are IID are collected and iterations are run to define each user data with random index. Here focuses are put on equal and random distribution of data across all users, while the NON-IID are assigned different data distributions to users based on class labels and shards. Available NON-IID image data are divided into 200 shards having 300 images each. An array is set consisting of the randomly given labels to the data where the IID was given equally distributed labels for all data. Then after preprocessing downloaded data from MNIST they distributed them to 100 users and assigned to 'd' the preparation for federal learning is done.

In fed.py **Fedavg** function is defined to average the weights applied in the training.

In net.py The neural network models are set up with setting up the input layer number, hidden layers and output layers.

To train globally all users are taken after loading all the data and splitting of them and then are assigned weights according to the labels with iterations. Using those weights, the training is to completed. Parallels the losses are computed. Weights applied are made average to send in **net_glob**. Also the losses are made average. Using this output average loss and average weights the test function is run to find out the accuracy level. In test.py the datasets and the **args** parameters are to be given, calculate the predictions and losses between the predictions and the true values. Then the number of correct predictions are to be found too. The average losses are to be taken, accuracy percentage too from test output. The server then creates a new version of the global model by averaging all the updates it received and sends it back to the devices for further training. After averaging the updates, the server sends the improved global model back to another randomly selected group of devices. The server doesn't use all devices in every round, which would be inefficient. Instead, it selects a small fraction of clients to participate in each round of updates.

3. Conclusion

In conclusion, the whole algorithm can be divided into two parts- improving locally training model on batches of data with this happening on the respective devices of the users and sending locally run output – weights and losses to the global model and as now the server knows where to improve train a percentage of all clients data centrally. That again sends the updates to the first part of this algorithm for local model to improve more and send update again to the global model until the whole training is complete for all the data.

References

McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. a. Y. (2016). Communication-Efficient Learning of Deep Networks from Decentralized Data. arXiv (Cornell University). <https://doi.org/10.48550/arxiv.1602.05629>